CHAPTER 14

CloudWatch, X-Ray and CloudTrail

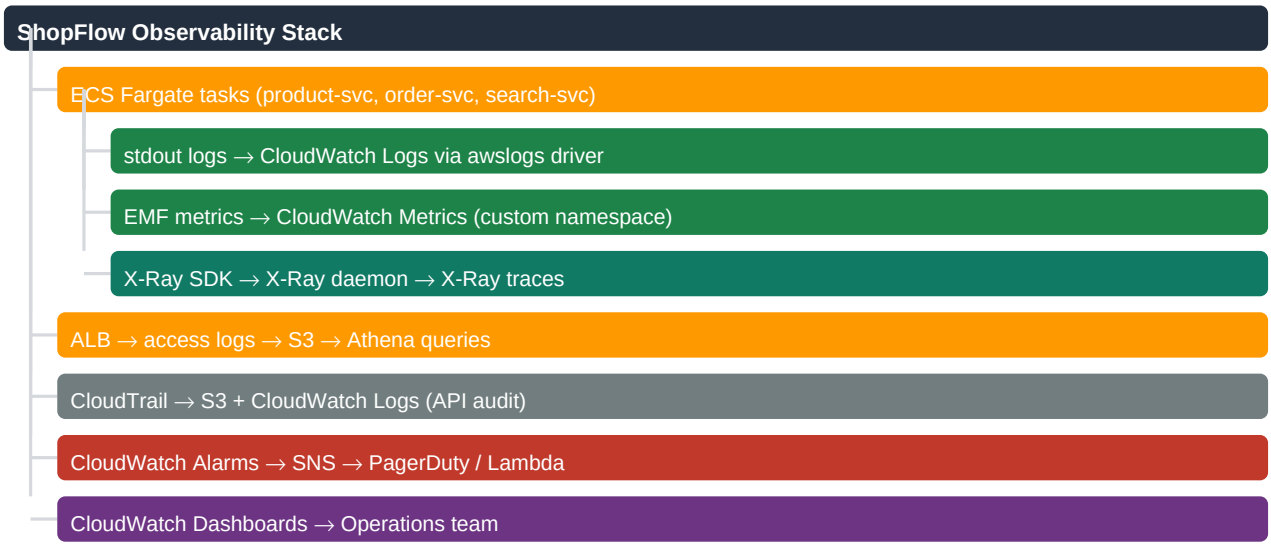
Metrics · Logs Insights · EMF · X-Ray Traces · CloudTrail · Dashboards

AWS Tutorial · ShopFlow

14-00 Chapter Overview — Observability at ShopFlow

Observability = Metrics + Logs + Traces. ShopFlow uses CloudWatch (metrics and logs), X-Ray (distributed traces), and CloudTrail (API audit). Together they answer: What is the system doing? (metrics), What happened? (logs), Why is it slow? (traces), Who changed what? (audit).

Pillar	Service	ShopFlow Use Case	Data Retention
Metrics	CloudWatch Metrics	ECS CPU/memory, ALB latency, Aurora connections, custom EMF metrics	15 months (default)
Logs	CloudWatch Logs	ECS task stdout/stderr, Lambda execution logs, ALB access logs (query)	30 days (configurable)
Traces	AWS X-Ray	End-to-end request tracing: ALB → ECS → Aurora → Redis → OpenSearch	30 days
Audit	AWS CloudTrail	All AWS API calls: who created/deleted/modified what, from which IP	7 years (S3 archive)
Dashboards	CloudWatch Dashboards	ShopFlow Operations Dashboard: latency, error rate, ECS task count, queue depth	N/A (live view)



14-01 CloudWatch Metrics — Key Metrics for ShopFlow

CloudWatch collects metrics from all AWS services automatically. Key ShopFlow metrics span ECS (task health), ALB (request rate and latency), Aurora (connections and latency), ElastiCache (cache hit rate), SQS (queue depth), and Lambda (errors and duration). Metric math expressions derive composite KPIs.

Service	Key Metric	Namespace	Alert Threshold
ECS Fargate	CPUUtilization, MemoryUtilization	AWS/ECS	CPU >80% for 5 min → scale out
ALB	TargetResponseTime, HTTPCode_Target_5XX_Count	AWS/ApplicationELB	p99 >2s; 5XX >10/min
Aurora PostgreSQL	DatabaseConnections, ReadLatency, WriteLatency	AWS/RDS	Connections >500; Write latency >100ms
ElastiCache Redis	CacheHitRate, CurrConnections, Evictions	AWS/ElastiCache	HitRate <80%; Evictions >0

Service	Key Metric	Namespace	Alert Threshold
SQS	ApproximateNumberOfMessagesVisible	AWS/SQS	Depth >1000 for ORDER_PROCESSING
Lambda	Errors, Duration, Throttles	AWS/Lambda	Errors >5/min; Throttles >0
OpenSearch	SearchLatency, JVMMemoryPressure	AWS/ES	Search p99 >200ms; JVM >85%

14-02 CloudWatch Logs — ECS Log Configuration

ECS Fargate sends container stdout/stderr to CloudWatch Logs via the awslogs log driver. Each service writes to its own log group (/shopflow/product-svc, /shopflow/order-svc). Structured JSON logging (Logback + logstash-logback-encoder) enables CloudWatch Logs Insights to parse fields without regex.

ECS Task Definition Log Configuration CDK

```
const taskDef = new ecs.FargateTaskDefinition(this,"ProductTaskDef",{...});
const container = taskDef.addContainer("product-svc",{
image: ecs.ContainerImage.fromEcrRepository(repo, imageTag),
logging: ecs.LogDrivers.awsLogs({
logGroup: new logs.LogGroup(this,"ProductLogGroup",{
logGroupName: "/shopflow/product-svc",
retention: logs.RetentionDays.ONE_MONTH, // 30-day retention
removalPolicy: RemovalPolicy.RETAIN, // keep logs if CDK stack deleted
}),
streamPrefix: "product-svc",
mode: ecs.AwsLogDriverMode.NON_BLOCKING, // never block app if CloudWatch slow
maxBufferSize: Size.mebibytes(25), // 25MB buffer for non-blocking mode
}),
});
```

Structured JSON Logging — Spring Boot Logback

```
// logback-spring.xml: JSON structured logs
<appender name="JSON" class="ch.qos.logback.core.ConsoleAppender">
<encoder class="net.logstash.logback.encoder.LogstashEncoder">
<fieldNames>
<timestamp>@timestamp</timestamp>
<version>[ignore]</version>
</fieldNames>
<customFields>{"service":"product-svc","env":"prod"}</customFields>
</encoder>
</appender>
// Output: {"@timestamp":"2024-11-29T10:00:00Z","level":"INFO",
// "message":"Product fetched","productId":"PROD-123",
// "latencyMs":45,"traceId":"1-abc-def","service":"product-svc"}
```

■ Use NON_BLOCKING log driver mode (ecs.AwsLogDriverMode.NON_BLOCKING). In blocking mode (default), if CloudWatch Logs becomes unavailable, container logging blocks app threads — causing request timeouts even when the application itself is healthy. Non-blocking mode uses an in-memory buffer (25MB); if the buffer fills, oldest logs are dropped (preferable to dropping requests). Dropped logs generate a CloudWatch metric: cwagent buffer_dropped_logs.

14-03 CloudWatch Logs Insights — Query Language

CloudWatch Logs Insights provides a SQL-like query language for analyzing log groups. ShopFlow uses Insights to: find slow product API calls, identify error spikes by service, compute p99 latency from structured JSON logs, and correlate errors with specific product IDs or user sessions.

CloudWatch Logs Insights Queries

```
// Query 1: p99 latency per endpoint (last 1 hour)
fields @timestamp, endpoint, latencyMs
```

```
| filter service = "product-svc" and level = "INFO"
| stats percentile(latencyMs, 99) as p99, count(*) as requests
by endpoint
| sort p99 desc
// Query 2: Errors in last 5 minutes with context
fields @timestamp, level, message, productId, traceId
| filter level = "ERROR" and service = "product-svc"
| sort @timestamp desc
| limit 50
// Query 3: Top 10 slowest product fetches (Black Friday analysis)
fields @timestamp, productId, latencyMs, userId
| filter service = "product-svc" and message = "Product fetched"
| sort latencyMs desc
| limit 10
// Query 4: Error rate % per service (last 30 minutes)
fields service, level
| filter level in ["ERROR", "WARN", "INFO"]
| stats count(*) as total,
sum(level = "ERROR") as errors
by service
| fields @timestamp, service, errors/total * 100 as errorRatePct
| sort errorRatePct desc
```

■ CloudWatch Logs Insights pricing: $0.005 per GB scanned. A 30-day log group for product-svc at 1,000 req/s × 500 bytes/log = 1.3TB/day × 30 days = 39TB. Querying 1 hour of logs: 1.3TB/24h × 1h = 54GB → $0.27 per query. To reduce scan cost: use narrow time ranges, filter on indexed fields (@timestamp, @logStream), and archive old logs to S3 (Logs Insights cannot query S3 — use Athena for S3 log analysis).

14-04 Custom Metrics with Embedded Metric Format (EMF)

EMF (Embedded Metric Format) publishes custom CloudWatch metrics from application code without the PutMetricData API call. EMF embeds metric data inside structured JSON log lines — CloudWatch Logs automatically extracts and publishes the metrics. Zero API calls, zero latency cost vs PutMetricData.

EMF Custom Metrics — Spring Boot

```
// Maven dependency: software.amazon.cloudwatch.emf:aws-embedded-metrics
@Service
public class ProductService {
private final MetricsLogger emf = MetricsLoggerFactory.createMetricsLogger();
public Product getProduct(String productId) {
long start = System.currentTimeMillis();
try {
Product product = repo.findById(productId).orElseThrow();
long latencyMs = System.currentTimeMillis() - start;
// Publish custom metric via EMF (embedded in log line)
emf.setNamespace("ShopFlow/ProductService");
emf.putDimensions(DimensionSet.of("Service","product-svc","Env","prod"));
emf.putMetric("ProductFetchLatency", latencyMs, Unit.MILLISECONDS);
emf.putMetric("ProductFetchSuccess", 1, Unit.COUNT);
emf.putProperty("productId", productId);
emf.flush();
return product;
} catch (Exception e) {
emf.putMetric("ProductFetchError", 1, Unit.COUNT);
emf.flush();
throw e;
}
}
}
```

Metric Method	Approach	Latency Cost	Cost
EMF (ShopFlow)	JSON log line → CW Logs extracts metric	~0ms (async log write)	$0 API calls + log storage
PutMetricData API	Direct SDK call to CloudWatch	~5-20ms per call	$0.01 per 1000 metrics
CloudWatch Agent	Agent reads app metric endpoint (Prometheus)	Agent polls every 60s	Agent EC2/ECS cost + metric cost
StatsD/collectd	Agent aggregates and sends	Agent aggregation delay	Agent cost + metric cost

14-05 CloudWatch Alarms and Composite Alarms

CloudWatch alarms trigger SNS notifications and Auto Scaling actions. Composite alarms combine multiple alarms with AND/OR logic — reduce alert fatigue by alerting only when multiple conditions are simultaneously true. ShopFlow: alert only when BOTH ALB 5XX rate is high AND ECS CPU is high (indicating overload, not a single task failure).

CloudWatch Alarm CDK — Composite Pattern

```
// Alarm 1: ALB 5XX errors
const alb5xxAlarm = new cloudwatch.Alarm(this,"Alb5xx",{
metric: alb.metricHttpCodeTarget(elbv2.HttpCodeTarget.TARGET_5XX_COUNT,
{period: Duration.minutes(1), statistic: "Sum"}),
threshold: 10, evaluationPeriods: 1,
alarmDescription: "ALB 5XX errors > 10/min",
treatMissingData: cloudwatch.TreatMissingData.NOT_BREACHING,
});
// Alarm 2: ECS CPU > 80%
const ecsCpuAlarm = new cloudwatch.Alarm(this,"EcsCpu",{
metric: ecsService.metricCpuUtilization({period: Duration.minutes(5)}),
threshold: 80, evaluationPeriods: 2, // 10 consecutive minutes
alarmDescription: "ECS CPU > 80% for 10 min",
});
// Composite Alarm: BOTH conditions must be true for critical alert
const criticalAlarm = new cloudwatch.CompositeAlarm(this,"Critical",{
compositeAlarmName: "shopflow-critical",
alarmRule: cloudwatch.AlarmRule.allOf(alb5xxAlarm, ecsCpuAlarm),
actionsEnabled: true,
});
criticalAlarm.addAlarmAction(new cwActions.SnsAction(pagerDutyTopic));
```

Alarm Type	Logic	Use Case	Alert Fatigue
Single alarm	One metric threshold	Quick spike detection	High — every transient spike alerts
Composite AND	All child alarms in ALARM	Overload: high CPU AND high errors together	Low — only true system issues page
Composite OR	Any child alarm in ALARM	Any severity event pages immediately	Medium — catches all issues fast
M of N evaluation	N periods, alarm if M are breaching	Sustained issues (not transient spikes)	Low — ignores momentary spikes

14-06 AWS X-Ray — Distributed Tracing

X-Ray traces each request through all microservices and AWS resources. ShopFlow configures X-Ray SDK in every Spring Boot service. X-Ray segments represent service calls; subsegments represent Aurora queries, Redis calls, and HTTP calls. The X-Ray Service Map shows the complete call graph with latency at each node.

X-Ray Configuration — Spring Boot

```
// pom.xml: X-Ray Spring Boot auto-configuration
<dependency>
<groupId>com.amazonaws</groupId>
<artifactId>aws-xray-recorder-sdk-spring</artifactId>
```

```
<version>2.15.3</version>
</dependency>
<dependency>
<groupId>com.amazonaws</groupId>
<artifactId>aws-xray-recorder-sdk-aws-sdk-v2-instrumentor</artifactId>
<version>2.15.3</version>
</dependency>
// application.yaml: enable X-Ray
aws.xray:
tracing.name: shopflow-product-svc
sampling:
rules-file: classpath:xray-sampling-rules.json
// ECS Task Definition: X-Ray daemon sidecar
taskDef.addContainer("xray-daemon",{
image: ecs.ContainerImage.fromRegistry("public.ecr.aws/xray/aws-xray-daemon:3.x"),
portMappings: [{containerPort: 2000, protocol: ecs.Protocol.UDP}],
cpu: 32, memoryLimitMiB: 256,
essential: false, // xray failure should not stop app container
logging: ecs.LogDrivers.awsLogs({logGroup: xrayLogGroup, streamPrefix: "xray"}),
});
```

■ Set X-Ray sampling rate to 5% in production for high-traffic services. At 1,000 req/s × 5% = 50 traces/s — sufficient for latency analysis without the cost of 100% sampling. For debugging specific endpoints (e.g., /api/orders POST), use sampling rules to trace 100% of a specific URL pattern while keeping other endpoints at 5%. X-Ray sampling rules are evaluated per-request at the SDK; the daemon forwards all captured traces.

14-07 X-Ray Service Map and Trace Analysis

X-Ray Service Map shows all services and their connections with average latency and error rate at each node. ShopFlow service map: ALB → product-svc → Aurora, Redis, OpenSearch, order-svc. Click any node to see traces through that service, filter by status code, or drill into individual slow traces.

X-Ray Feature	What It Shows	ShopFlow Use Case
Service Map	Graph of all services and connections; latency + error rate per edge	Identify which service is causing latency (Aurora vs Redis vs Oper
Trace Timeline	Gantt chart of all subsegments for one request	Diagnose why a specific product page took 800ms (Aurora: 650m
Trace Groups	Filter traces by annotation (product category, user tier)	Separate traces for free vs premium users; compare latency
Analytics	Aggregate statistics across trace samples	Compute p99 Aurora query latency from X-Ray data (not CloudW
Insights	Auto-detected anomalies (latency spike, error surge)	Alert when X-Ray detects a service degradation automatically

Adding Custom Annotations to Traces

```
// Add annotations (filterable in X-Ray console) and metadata to traces
@RestController
public class ProductController {
@GetMapping("/api/products/{id}")
public Product getProduct(@PathVariable String id) {
// Add annotation: filterable in X-Ray (indexed)
AWSXRay.getCurrentSegment().putAnnotation("productId", id);
AWSXRay.getCurrentSegment().putAnnotation("userId",
SecurityContextHolder.getContext().getAuthentication().getName());
// Add metadata: stored but not filterable (for debugging)
AWSXRay.getCurrentSegment().putMetadata("request","queryParams",
request.getParameterMap());
return productService.getProduct(id);
}
}
```

14-08 AWS CloudTrail — API Audit Logging

CloudTrail records every AWS API call: who called which API, from which IP, at what time, with what parameters, and what the response was. ShopFlow uses CloudTrail for: security compliance (who deleted the S3 bucket?), change management (who modified the ECS task definition?), and forensics (unauthorized IAM role change?).

CloudTrail CDK — Organization Trail

```
const trail = new cloudtrail.Trail(this,"ShopFlowTrail",{
trailName: "shopflow-audit-trail",
bucket: auditBucket,
includeGlobalServiceEvents: true, // IAM, STS (global APIs)
isMultiRegionTrail: true, // all regions (catch us-west-2 DR activity)
enableFileValidation: true, // log file integrity (SHA-256 digest)
sendToCloudWatchLogs: true, // real-time alerting on specific API calls
cloudWatchLogsGroup: auditLogGroup,
managementEvents: cloudtrail.ReadWriteType.ALL, // management plane events
dataEvents: [
{
resources: [{type: "AWS::S3::Object",
values: [`${assetsBucket.bucketArn}/`]}], // product images S3 bucket
readWriteType: cloudtrail.ReadWriteType.WRITE_ONLY, // only writes (deletes)
}
],
});
```

CloudTrail Event Type	Examples	ShopFlow Config	Cost
Management events	CreateBucket, TerminateInstances, CreateRole, PutBucketPolicy	All (read+write) — included free in first trail	Free for 1 trail
Data events (S3)	GetObject, PutObject, DeleteObject on specific buckets	WRITE_ONLY on assets bucket — deletes only	$0.10 per 100K events
Data events (Lambda)	InvokeFunction — every Lambda invocation logged	Not enabled — too high volume, cost	$0.10 per 100K events
CloudTrail Insights	Unusual API activity patterns (spike in TerminateInstances)	Not enabled (cost $0.35/100K events)	$0.35 per 100K events

■ Never delete the CloudTrail S3 bucket from an admin console. Enable MFA Delete on the CloudTrail S3 bucket (requires root account confirmation for deletion) and S3 Object Lock (COMPLIANCE mode) for 7-year retention. CloudTrail file validation (SHA-256 digest per log file) detects if log files were tampered after delivery to S3. Forensic integrity requires: (1) Object Lock prevents deletion, (2) file validation detects modification.

14-09 CloudWatch Dashboards for Operations

CloudWatch Dashboards display real-time metrics as widgets. ShopFlow Operations Dashboard consolidates all critical KPIs in one view: request rate, error rate, p99 latency, ECS task count, Aurora connections, Redis hit rate, and SQS queue depth. Dashboards are shared read-only with stakeholders.

Dashboard CDK — Key Widgets

```
const dashboard = new cloudwatch.Dashboard(this,"OpsDashboard",{
dashboardName: "ShopFlow-Operations",
defaultInterval: Duration.hours(1),
});
dashboard.addWidgets(
// Row 1: Traffic overview
new cloudwatch.GraphWidget({
title: "ALB Request Rate + Error Rate",
left: [alb.metricRequestCount({period: Duration.minutes(1)})],
right: [alb.metricHttpCodeTarget(elbv2.HttpCodeTarget.TARGET_5XX_COUNT,
{period: Duration.minutes(1), statistic: "Sum"})],
width: 12, height: 6,
}),
new cloudwatch.GraphWidget({
```

```
title: "ALB Latency (p50, p95, p99)",
left: [alb.metricTargetResponseTime({statistic:"p50"}),
alb.metricTargetResponseTime({statistic:"p95"}),
alb.metricTargetResponseTime({statistic:"p99"})],
width: 12, height: 6,
}),
// Row 2: ECS + Aurora
new cloudwatch.GraphWidget({
title: "ECS CPU + Memory",
left: [ecsService.metricCpuUtilization(),
ecsService.metricMemoryUtilization()],
width: 12, height: 6,
}),
);
```

14-10 Log Metric Filters — Logs to Metrics

CloudWatch Log Metric Filters extract numeric values or count occurrences from log lines and publish them as CloudWatch metrics. ShopFlow uses metric filters to: count ERROR log lines per minute, extract productId from error logs for product-specific error rate, and detect specific exception types (SocketTimeoutException).

Log Metric Filter CDK

```
// Count ERROR lines per minute from product-svc logs
const errorFilter = new logs.MetricFilter(this,"ProductErrorMetric",{
logGroup: productLogGroup,
filterPattern: logs.FilterPattern.stringValue(
"$.level", "=", "ERROR"), // JSON log field: level == "ERROR"
metricNamespace: "ShopFlow/ProductService",
metricName: "ErrorCount",
metricValue: "1", // count each match as 1
unit: cloudwatch.Unit.COUNT,
defaultValue: 0, // when no matches: publish 0 (keeps alarm from INSUFFICIENT_DATA)
});
// Extract latency values from log lines for p99 calculation
const latencyFilter = new logs.MetricFilter(this,"ProductLatencyMetric",{
logGroup: productLogGroup,
filterPattern: logs.FilterPattern.numberValue("$.latencyMs", ">=", 0),
metricNamespace: "ShopFlow/ProductService",
metricName: "RequestLatencyMs",
metricValue: "$.latencyMs", // extract actual value from JSON field
unit: cloudwatch.Unit.MILLISECONDS,
});
```

■ Always set defaultValue: 0 on metric filters that count errors or events. Without defaultValue, if no matching log lines occur in a period, CloudWatch publishes no data point — alarms enter INSUFFICIENT_DATA state (not ALARM or OK). An alarm in INSUFFICIENT_DATA does not trigger when the error rate drops to zero (no logs = no metric = no data). With defaultValue: 0, zero errors publish a 0 data point, keeping the alarm in OK state correctly.

14-11 CloudWatch Container Insights

Container Insights provides enhanced monitoring for ECS Fargate: per-task CPU, memory, network, and storage metrics at the task level (not just the service average). Enabled per ECS cluster. ShopFlow uses Container Insights to identify which specific Fargate tasks are using excessive memory (potential memory leak detection).

Enable Container Insights CDK

```
const cluster = new ecs.Cluster(this,"ShopFlowCluster",{
vpc,
containerInsights: true, // enables Container Insights on all services
});
// Container Insights metrics (additional namespace: ECS/ContainerInsights):
```

```
// TaskCount, RunningTaskCount, PendingTaskCount
// CpuUtilized, CpuReserved, MemoryUtilized, MemoryReserved
// NetworkRxBytes, NetworkTxBytes
// StorageReadBytes, StorageWriteBytes
// EphemeralStorageUtilized, EphemeralStorageReserved
```

Monitoring Level	Metrics Available	ShopFlow Use
Service level (default)	CPUUtilization, MemoryUtilization (avg across all tasks)	Scale-out triggers (avg CPU > 70%)
Task level (Container Insights)	Per-task CPU, memory, network — identify outlier tasks	Detect memory leak: one task at 90% while others at 40
Container level (Container Insights)	Per-container metrics within multi-container tasks	Separate app vs X-Ray sidecar resource use

Detect Memory Leak with Container Insights Query

```
// CloudWatch Metrics Insights query: find tasks with > 80% memory
// Namespace: ECS/ContainerInsights, Metric: MemoryUtilized
// Dimension: TaskId (per-task granularity)
// Alert: if any individual task exceeds 80% while service avg is < 50%,
// it indicates a memory leak in that specific task.
// Action: ECS stop-task API terminates the leaking task;
// ECS service scheduler immediately replaces it with a new task.
```

14-12 CloudWatch Synthetics — Canary Monitoring

CloudWatch Synthetics runs canary scripts (Node.js/Python) on a schedule to test endpoints from outside the VPC — simulating real user flows. ShopFlow canaries: (1) homepage load test every 5 min, (2) product search canary every 1 min, (3) checkout flow canary every 15 min. Canary failures trigger PagerDuty before real users notice.

Synthetics Canary CDK

```
const searchCanary = new synthetics.Canary(this,"SearchCanary",{
canaryName: "shopflow-search-canary",
schedule: synthetics.Schedule.rate(Duration.minutes(1)),
runtime: synthetics.Runtime.SYNTHETICS_NODEJS_PUPPETEER_6_2,
test: synthetics.Test.custom({
code: synthetics.Code.fromAsset("./canaries/search-canary.js"),
handler: "index.handler",
}),
vpc,
vpcSubnets: {subnetType: ec2.SubnetType.PRIVATE_WITH_EGRESS},
artifactsBucketLocation: {bucket: canaryBucket},
environmentVariables: {
SEARCH_URL: "https://api.shopflow.com/search?q=laptop",
EXPECTED_MIN_RESULTS: "5",
},
alarmSnsTopic: alertTopic, // alert if canary fails 2 consecutive runs
});
```

Canary	Schedule	Test Flow	Alert Threshold
Homepage canary	Every 5 minutes	GET shopflow.com; verify HTTP 200; screenshot	2 consecutive failures
Search canary	Every 1 minute	GET /search?q=laptop; verify >5 results; latency <500ms	1 failure (critical endpoint)
Add-to-cart canary	Every 15 minutes	POST /api/cart; verify 201; delete cart item	2 consecutive failures
Checkout canary	Every 30 minutes	Full checkout flow with test credit card (Stripe test mode)	1 failure (revenue-critical)

■ Run canaries from inside the VPC (vpc config) to test internal endpoints directly. If the canary runs outside the VPC, it tests through ALB/WAF and may be blocked by WAF rate-limiting rules when canary runs every 1 minute. Inside the VPC, canaries can test ALB at its private IP, bypassing WAF. For public-facing canary (testing WAF, TLS, DNS): run outside VPC and whitelist the canary Lambda security group IP in WAF.

14-13 CloudWatch Alarms for Auto Scaling

CloudWatch Alarms are the trigger mechanism for ECS Auto Scaling. When CPUUtilization exceeds 70% for 2 consecutive 1-minute periods, a CloudWatch Alarm fires an SNS notification that triggers the ECS Application Auto Scaling policy. Scale-in: when CPU drops below 30% for 10 consecutive minutes (to avoid flapping).

Auto Scaling Alarm CDK

```
const scalableTarget = service.autoScaleTaskCount({
minCapacity: 3,
maxCapacity: 50,
});
// Scale OUT: CPU > 70% for 2 minutes
scalableTarget.scaleOnMetric("CpuScaleOut",{
metric: service.metricCpuUtilization({period: Duration.minutes(1)}),
scalingSteps: [
{lower: 70, change: +2}, // add 2 tasks when 70-85%
{lower: 85, change: +5}, // add 5 tasks when > 85%
],
cooldown: Duration.seconds(60),
adjustmentType: ScalingAdjustmentType.CHANGE_IN_CAPACITY,
});
// Scale IN: CPU < 30% for 10 minutes (conservative — avoid flapping)
scalableTarget.scaleOnMetric("CpuScaleIn",{
metric: service.metricCpuUtilization({period: Duration.minutes(1)}),
scalingSteps: [{upper: 30, change: -1}],
cooldown: Duration.seconds(600), // 10-min cooldown on scale-in
adjustmentType: ScalingAdjustmentType.CHANGE_IN_CAPACITY,
});
```

Alarm Parameter	Scale-Out Config	Scale-In Config	Rationale
Threshold	70% CPU	30% CPU	Asymmetric: scale out fast, scale in conservatively
Evaluation Periods	2 periods (2 min)	10 periods (10 min)	Scale in slowly: avoid oscillation (flapping)
Period	1 minute	1 minute	1-minute granularity for fast response
Cooldown	60 seconds	600 seconds	Short cooldown for scale-out; long for scale-in

14-14 CloudWatch Logs Metric Filters for SLA Tracking

ShopFlow SLA: API p99 latency <2s; availability >99.9% (8.7 hours downtime/year). Log metric filters extract request durations from structured logs to build SLA compliance metrics. A daily Lambda function queries the past 24h latency metric and publishes SLA compliance percentage to a compliance dashboard.

SLA Compliance Metric Filter

```
// Metric filter: count requests exceeding 2s SLA
const slaBreachFilter = new logs.MetricFilter(this,"SlaBreach",{
logGroup: productLogGroup,
filterPattern: logs.FilterPattern.all(
logs.FilterPattern.stringValue("$.level","=","INFO"),
logs.FilterPattern.numberValue("$.latencyMs",">=",2000)),
metricNamespace: "ShopFlow/SLA",
metricName: "RequestsExceedingSLA",
metricValue: "1",
defaultValue: 0,
});
// SLA compliance % = (TotalRequests - SLABreaches) / TotalRequests * 100
// Metric Math expression in CloudWatch Dashboard:
// e1 = TotalRequests (from request count metric filter)
```

```
// e2 = RequestsExceedingSLA
// SLA% = (e1 - e2) / e1 * 100
```

SLA Metric	Target	Measurement Method	Alert If
API p99 latency	<2s	CloudWatch: ALB TargetResponseTime p99	p99 >2s for 5 min
API availability	>99.9%	CloudWatch: ALB RequestCount vs 5XX count	Error rate >0.1% for 5 min
Search latency	<200ms	EMF: ShopFlow/Search SearchLatencyMs	p99 >200ms for 3 min
Order processing time	<30s end-to-end	X-Ray trace: checkout start to order confirmation	Avg >30s for 10 min
Autocomplete latency	<50ms	EMF: ShopFlow/Search AutocompleteLatencyMs	p99 >50ms for 2 min

14-15 CloudWatch Logs Export to S3 and Athena

CloudWatch Logs are stored at $0.03/GB/month. For long-term retention (compliance: 7 years), export logs to S3 ($0.023/GB/month) and query with Athena. Export is asynchronous: CloudWatch creates a zip file in S3 within 12 hours. For real-time S3 export, use Kinesis Data Firehose subscription filter.

Log Export to S3 via Firehose Subscription Filter

```
// Real-time: CloudWatch Logs → Kinesis Firehose → S3
const logSubscription = new logs.CfnSubscriptionFilter(this,"LogSub",{
logGroupName: productLogGroup.logGroupName,
filterPattern: "", // all log events
destinationArn: firehose.deliveryStreamArn,
roleArn: cwToFirehoseRole.roleArn,
});
// Firehose delivers to: s3://shopflow-logs-archive/product-svc/year=2024/month=11/
// Athena query over S3:
// CREATE EXTERNAL TABLE product_logs (
// timestamp STRING, level STRING, message STRING,
// productId STRING, latencyMs BIGINT, traceId STRING)
// PARTITIONED BY (year STRING, month STRING)
// STORED AS OPENX_JSON LOCATION "s3://shopflow-logs-archive/product-svc/"
```

Method	Latency	Cost (storage)	Query Engine	ShopFlow Use
CW Logs (30 days)	Real-time	$0.03/GB/mo	CW Logs Insights ($0.005/GB scanned)	Operational: last 30 days of logs
S3 export (async)	Up to 12 hours	$0.023/GB/mo	Athena ($5/TB scanned)	Compliance archive: 7-year retention
S3 via Firehose (real-time)	60s buffer	$0.023/GB/mo	Athena	Real-time S3 pipeline for audit compliance

14-16 CloudWatch Anomaly Detection

CloudWatch Anomaly Detection uses machine learning to automatically compute expected metric values (band) based on historical patterns. Alarms fire when a metric falls outside the band. ShopFlow uses Anomaly Detection for: daily request rate (traffic drops on weekend), checkout error rate, and Aurora connection count (normal variation by hour of day).

Anomaly Detection Alarm CDK

```
// Create anomaly detector for ALB request count
const anomalyDetector = new cloudwatch.CfnAnomalyDetector(this,"ReqAnomaly",{
namespace: "AWS/ApplicationELB",
metricName: "RequestCount",
stat: "Sum",
dimensions: [{name:"LoadBalancer",value:alb.loadBalancerFullName}],
configuration: {
excludedTimeRanges: [ // exclude Black Friday from training data
{startTime:"2024-11-29T00:00:00Z",endTime:"2024-11-30T00:00:00Z"}
```

```
]
}
});
// Alarm: fire if requests fall > 2 standard deviations below expected band
// (traffic drop = possible DNS issue or application crash)
const trafficDropAlarm = new cloudwatch.Alarm(this,"TrafficDrop",{
metric: alb.metricRequestCount({period: Duration.minutes(5)}),
comparisonOperator: cloudwatch.ComparisonOperator.LESS_THAN_LOWER_THRESHOLD,
evaluationPeriods: 3,
threshold: 0, // threshold unused for anomaly detection alarms
});
```

■ Anomaly Detection training period: CloudWatch collects at least 2 weeks of historical data before the anomaly detector produces reliable bands. The model learns: daily patterns (peak at noon, trough at 3am), weekly patterns (lower on weekends), and excludes specified time ranges (Black Friday spike). Cost: $0.30 per alarm per month for Anomaly Detection alarms (3x the standard $0.10/alarm).

14-17 X-Ray Groups and Sampling Rules

X-Ray Groups filter traces by expression (service name, URL, response code). ShopFlow defines groups for: slow traces (ResponseTime > 1s), error traces (fault = true), and checkout traces (URL contains /checkout). Each group has its own CloudWatch metric (trace count, error rate) that can drive alarms.

X-Ray Sampling Rules CDK

```
// Sampling rule: sample 100% of checkout requests (revenue-critical)
new xray.CfnSamplingRule(this,"CheckoutSampling",{
samplingRule: {
ruleName: "checkout-full-sample",
priority: 1, // lowest number = highest priority
reservoir: 1, // 1 trace/s guaranteed regardless of rate
fixedRate: 1.0, // 100% of checkout requests traced
httpMethod: "POST",
urlPath: "/api/orders*",
serviceName: "shopflow-product-svc",
serviceType: "*",
host: "*",
resourceArn: "*",
}
});
// Default rule: 5% sampling for all other requests
// priority 9999 (default, lowest priority) fixedRate 0.05
```

Sampling Rule	Priority	Rate	ShopFlow Purpose
checkout-full-sample	1	100%	Trace every order creation — no missed checkout failures
health-check-exclude	5	0%	Exclude /actuator/health — 1000s of traces/min, no value
search-reduced	10	1%	Search: 1000 req/s × 1% = 10 traces/s — sufficient for analysis
error-capture	20	100% of 5XX	Trace all errors — even at low overall sampling rate
default	9999	5%	All other requests: 1000 req/s × 5% = 50 traces/s

14-18 CloudTrail Lake for Advanced Audit Queries

CloudTrail Lake is a managed data lake for CloudTrail events with SQL query support. Unlike CloudTrail + Athena (query S3), CloudTrail Lake stores events natively and supports complex JOINs across event types. ShopFlow uses CloudTrail Lake for: who changed which IAM policy in the last 90 days, cross-account resource access, and compliance reports.

CloudTrail Lake Query Examples

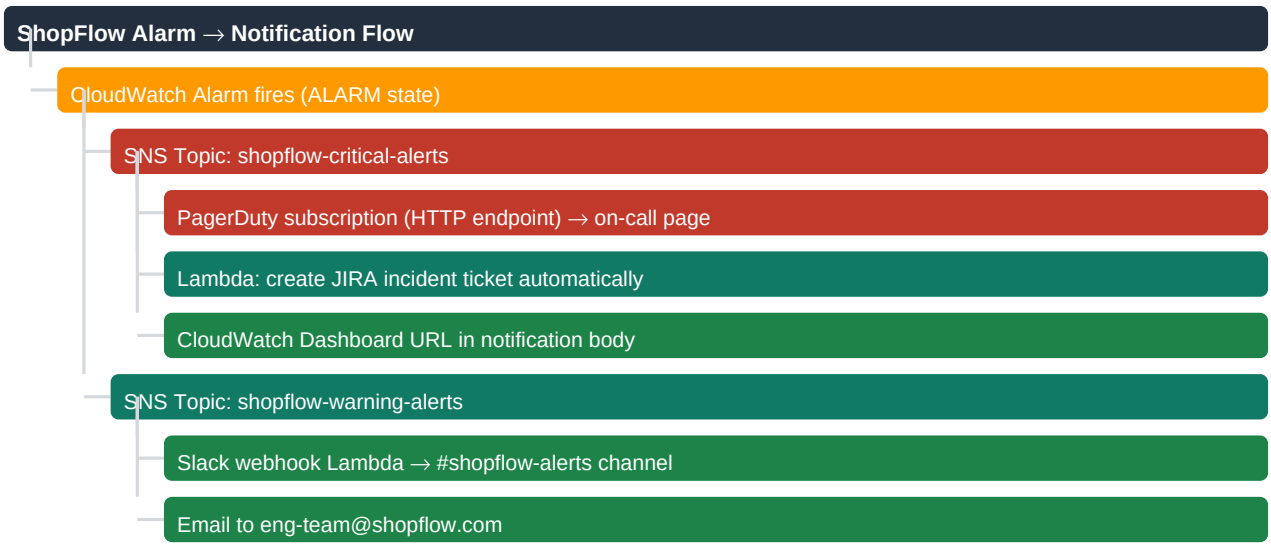
```
-- Query 1: Who deleted S3 objects from the assets bucket in the last 7 days?
SELECT userIdentity.principalId, eventTime, requestParameters
FROM cloudtrail_lake_event_store_id
WHERE eventSource = "s3.amazonaws.com"
AND eventName = "DeleteObject"
AND requestParameters LIKE "%shopflow-assets%"
AND eventTime > CURRENT_TIMESTAMP - INTERVAL 7 DAY
ORDER BY eventTime DESC
-- Query 2: IAM policy changes in the last 90 days
SELECT eventTime, userIdentity.principalId, requestParameters.policyArn
FROM cloudtrail_lake_event_store_id
WHERE eventSource = "iam.amazonaws.com"
AND eventName IN ("PutRolePolicy","AttachRolePolicy","DeleteRolePolicy")
AND eventTime > CURRENT_TIMESTAMP - INTERVAL 90 DAY
ORDER BY eventTime DESC
```

Feature	CloudTrail + Athena	CloudTrail Lake
Storage	S3 (user-managed)	Managed (CloudTrail Lake)
Query engine	Athena SQL	SQL (CloudTrail Lake native)
Setup	Complex (Glue catalog, partitions)	Simple (enable on trail)
Cross-account queries	Complex (cross-account S3 + Athena)	Built-in (multi-account support)
Retention	S3 lifecycle (user-controlled)	Up to 7 years (configurable)
Cost	S3 storage + Athena $5/TB	$0.75/GB ingestion + $0.005/GB query

---

14-19 **Alarm Notification Architecture**

CloudWatch Alarms publish to SNS topics when they transition to ALARM state. ShopFlow uses a multi-tier notification strategy: critical alarms (revenue-impacting) → PagerDuty (wakes on-call engineer), warning alarms → Slack channel (visible during business hours), informational events → email digest (daily summary). SNS fanout enables multiple destinations from one alarm.



SNS + Lambda Slack Notification CDK

```
// Lambda: formats CloudWatch alarm JSON → Slack Block Kit message
const slackFn = new lambda.Function(this,"SlackNotifier",{
runtime: lambda.Runtime.NODEJS_20_X,
```

```
handler: "index.handler",
code: lambda.Code.fromAsset("./lambdas/slack-notifier"),
environment: {
SLACK_WEBHOOK_URL: slackWebhookSecret.secretValue.toString(),
DASHBOARD_URL:
"https://console.aws.amazon.com/cloudwatch/home#dashboards:name=ShopFlow-Operations",
},
});
warnTopic.addSubscription(new snsSubscriptions.LambdaSubscription(slackFn));
```

Severity	Alarms	Destination	Response SLA
Critical (P1)	ALB 5XX > 50/min, ECS tasks = 0, Aurora down	PagerDuty + JIRA	5 minutes (auto-page on-call)
High (P2)	ALB p99 > 2s, Redis evictions, SQS depth > 5000	PagerDuty (low urgency) + Slack	15 minutes
Warning (P3)	ECS CPU > 80%, OpenSearch JVM > 85%	Slack #alerts + email	1 hour (business hours)
Info (P4)	Deployment started/completed, auto-scaling events	Slack #deployments	None (informational)

14-20 Cost of Observability at ShopFlow

Observability has a real cost. ShopFlow monthly observability budget: ~$850/month. Key costs: CloudWatch Logs ingestion ($0.50/GB), metrics ($0.30 per custom metric), X-Ray traces ($5/million), and Container Insights. Optimization: reduce log verbosity in INFO level, use sampling for X-Ray, archive old logs to S3.

Service	Usage	Unit Price	Monthly Cost
CloudWatch Logs ingestion	500 GB (all services)	$0.50/GB	$250
CloudWatch Logs storage	500 GB (30-day retention)	$0.03/GB	$15
CloudWatch custom metrics	100 custom metrics (EMF)	$0.30/metric	$30
CloudWatch alarms	50 alarms	$0.10/alarm	$5
CloudWatch Logs Insights	200 GB scanned/month	$0.005/GB	$1
X-Ray traces	50M traces/month (5% of 1B requests)	$5/million	$250
Container Insights	50 ECS tasks	$0.05/task/day x 30	$75
CloudTrail	Management events (1 trail free)	Free + S3 storage	$15 (S3)
Synthetics canaries	3 canaries x 720 runs/month	$0.0012/run	$2.60
Total			~$644/month

Cost Optimization — Reduce Log Volume

```
// Set INFO level for production (not DEBUG)
// application.yaml:
logging:
level:
root: WARN // only warnings and errors for libraries
com.shopflow: INFO // INFO for ShopFlow code
org.springframework.web: WARN // suppress Spring request logging
org.hibernate.SQL: WARN // suppress SQL query logging in prod
// Result: ~70% log volume reduction vs DEBUG level = $175/month savings
```

14-21 Event-Driven Alerting with EventBridge and CloudTrail

EventBridge can react to CloudTrail API events in near-real-time (within 15 minutes). ShopFlow uses EventBridge rules to: auto-remediate security issues (unauthorized IAM role creation triggers Lambda to delete the role), alert on cost anomalies (Budget threshold breach → SNS), and notify on ECS task failures.

EventBridge Rule for Unauthorized IAM Activity

```
// EventBridge rule: alert + remediate on IAM role creation outside CDK
new events.Rule(this,"UnauthorizedIamAlert",{
eventPattern: {
source: ["aws.iam"],
detailType: ["AWS API Call via CloudTrail"],
detail: {
eventSource: ["iam.amazonaws.com"],
eventName: ["CreateRole","AttachRolePolicy","CreateUser"],
userIdentity: {
type: [{"anything-but": "AssumedRole"}] // not from CI/CD role
}
}
},
targets: [
new targets.SnsTopic(securityAlertTopic), // alert security team
new targets.LambdaFunction(iamRemediateFn), // auto-delete the IAM role
],
});
```

EventBridge Trigger	Event Source	ShopFlow Action	Response Time
Unauthorized IAM role creation	CloudTrail: CreateRole	Alert security + Lambda auto-deletes role	<15 min (CloudTrail delay)
ECS task stopped (non-zero exit)	ECS: Task State Change	SNS alert; Slack message with task logs link	<1 min (real-time)
AWS Budget 80% threshold	AWS Budgets	SNS → Slack #finops channel	Real-time (Budget event)
Aurora failover completed	RDS: DB Instance Event	Notify ops; verify read replica promotion	Real-time (RDS event)
CodePipeline deployment failed	CodePipeline: Pipeline Execution	Page on-call; link to failed CodeBuild logs	Real-time (Pipeline event)

■ EventBridge receives CloudTrail events with up to 15-minute delay (CloudTrail delivers events to EventBridge within ~15 minutes of the API call). For real-time security alerts (e.g., detect unauthorized root login immediately), use CloudTrail + CloudWatch Logs + Metric Filter + Alarm — metric filters process log events as they arrive (seconds), not 15 minutes. EventBridge is better for complex pattern matching across multiple event types.

14-BP Best Practices and Anti-Patterns

✓ Use structured JSON logging (Logback + LogstashEncoder) — enables CloudWatch Logs Insights field queries without regex parsing

✓ Use EMF for custom metrics — zero latency, zero PutMetricData API calls, metrics embedded in log lines

✓ Use NON_BLOCKING awsLogs mode — prevents CloudWatch unavailability from blocking application threads

✓ Set log retention policy (30-90 days) on every log group — unlimited retention = unbounded cost

✓ Add X-Ray annotations for filterable trace search (productId, userId) — metadata is not searchable

✓ Use Composite Alarms (AND logic) to reduce alert fatigue — alert only when multiple KPIs degrade simultaneously

✓ Enable CloudTrail file integrity validation and S3 Object Lock (COMPLIANCE mode) for immutable audit logs

✓ Set defaultValue=0 on Log Metric Filters to prevent INSUFFICIENT_DATA alarm state when no log lines match

✓ Use Container Insights per-task metrics for memory leak detection — service-level averages hide individual task anomalies

✓ Sample X-Ray at 5% in production — 100% sampling at 1000 req/s = $5/day in X-Ray data charges vs $0.25/day at 5%

✓ Create CloudWatch dashboard per team (Ops, Dev, Business) — filter noise; show only team-relevant metrics

✓ Use CloudWatch Logs Insights for ad-hoc queries; build dashboards on CloudWatch Metrics (not Insights) for real-time alerts

■ Never use synchronous PutMetricData in the hot request path — 5-20ms latency per call; use EMF or CloudWatch Agent instead

■ Do not enable CloudTrail data events on all S3 buckets without cost analysis — $0.10/100K events; high-traffic buckets generate millions of events/day

■ Avoid creating too many CloudWatch alarms (> 500) — each alarm costs $0.10/month; composite alarms count as 1

■ Do not share CloudWatch dashboards with read/write access — use read-only share links to prevent accidental modification

14-QR Quick Reference and Interview Q&A

Concept	Key Value / Rule
Three pillars of observability	Metrics (what is happening — numeric time series), Logs (what happened — text events), Traces (why —
EMF vs PutMetricData	EMF: metric embedded in log line → CW Logs extracts → publishes metric. Zero API calls, zero latency in
X-Ray sampling rate	Default: 5% after first request per second (first req always sampled). At 1000 req/s: 1 + (999 × 0.05) = ~51
CloudTrail vs CloudWatch Logs	CloudTrail: AWS API calls (control plane) — who changed what resource. CloudWatch Logs: application a
Log metric filter defaultValue	Without: no log lines = no metric data point = alarm enters INSUFFICIENT_DATA (neither ALARM nor OK
Container Insights cost	~$0.05/task/day (ECS Fargate). 50 tasks × $0.05 × 30 days = $75/month additional cost. Standard ECS m